

Spec Driven Development – Plataforma PIX

Assíncrona

1. Contexto

A plataforma atual processa transações PIX de forma síncrona, passando por banco de dados, aplicação legada, core transacional e instituição financeira parceira.

O objetivo da solução é reduzir o impacto da latência externa, aumentar a capacidade de processamento, melhorar resiliência e garantir idempotência.

Stack definida:

- Java 21
- Spring Boot
- PostgreSQL
- Kafka
- Redis
- Resilience4j
- Datadog
- Docker Compose

Fase 1 — Arquitetura alvo

Serviços

A solução será composta por dois serviços principais:

```
pix-api  
pix-processor
```

Além deles, teremos componentes de infraestrutura:

```
PostgreSQL  
Kafka  
Redis  
WireMock  
Datadog Agent
```

Diagrama macro

```
Cliente
|
| POST /pix
v
+-----+
|  pix-api  |
+-----+
|           |
|           | grava transação + outbox
|           | v
|      PostgreSQL
|
| publica evento via outbox publisher
v
Kafka topic: pix.requested
|
v
+-----+
| pix-processor |
+-----+
|
| HTTP call
v
WireMock / Partner API
```

Fase 2 — Contratos de API

POST /pix

Responsável por receber uma solicitação de PIX.

Request

```
{
  "transactionId": "tx-123456",
  "amount": 150.75,
  "pixKey": "cliente@email.com",
  "description": "Pagamento de fatura"
}
```

Response

202 Accepted

```
{
  "transactionId": "tx-123456",
  "status": "PROCESSING",
  "createdAt": "2026-06-12T15:00:00Z"
}
```

GET /pix/{transactionId}

Consulta o status atual da transação.

Response

```
{
  "transactionId": "tx-123456",
  "status": "PROCESSING",
  "createdAt": "2026-06-12T15:00:00Z",
  "updatedAt": "2026-06-12T15:00:05Z"
}
```

Fase 3 — Modelagem de domínio

Entidade: PixTransaction

```
transactionId
amount
pixKey
description
status
partnerReferenceId
failureReason
createdAt
updatedAt
```

Status possíveis

```
PROCESSING
RETRYING
```

COMPLETED
FAILED

Tabela: pix_transaction

```
CREATE TABLE pix_transaction (  
  id BIGSERIAL PRIMARY KEY,  
  transaction_id VARCHAR(100) NOT NULL UNIQUE,  
  amount NUMERIC(19, 2) NOT NULL,  
  pix_key VARCHAR(255) NOT NULL,  
  description VARCHAR(255),  
  status VARCHAR(30) NOT NULL,  
  partner_reference_id VARCHAR(100),  
  failure_reason TEXT,  
  created_at TIMESTAMP NOT NULL,  
  updated_at TIMESTAMP NOT NULL  
);
```

Fase 4 — Idempotência

A idempotência será garantida principalmente pelo `transactionId`.

Regra

Se o mesmo `transactionId` for enviado novamente:

- não cria nova transação;
- não publica novo evento;
- retorna o status atual da transação existente.

Implementação

No PostgreSQL:

```
transaction_id UNIQUE
```

No Redis, opcionalmente:

```
pix:idempotency:tx-123456 -> PROCESSING
```

O Redis pode reduzir pressão no banco em cenários de alto volume, mas o banco continua sendo a fonte da verdade.

Trade-off

Redis melhora performance, mas adiciona complexidade e risco de inconsistência temporária. Por isso, ele será usado como cache auxiliar, não como fonte primária.

Fase 5 — Outbox Pattern

Para evitar inconsistência entre banco e Kafka, será usado Transactional Outbox.

Problema evitado

```
Salvou transação no banco
Falhou ao publicar evento no Kafka
Resultado: transação nunca processada
```

Solução

Na mesma transação do banco:

1. Salva pix_transaction
2. Salva outbox_event

Depois, um job publica os eventos pendentes no Kafka.

Tabela: outbox_event

```
CREATE TABLE outbox_event (
  id UUID PRIMARY KEY,
  aggregate_id VARCHAR(100) NOT NULL,
  event_type VARCHAR(100) NOT NULL,
  payload JSONB NOT NULL,
  status VARCHAR(30) NOT NULL,
  created_at TIMESTAMP NOT NULL,
  published_at TIMESTAMP
);
```

Fase 6 — Kafka

Tópicos

```
pix.requested  
pix.dlq
```

Evento PixRequested

```
{  
  "transactionId": "tx-123456",  
  "amount": 150.75,  
  "pixKey": "cliente@email.com",  
  "description": "Pagamento de fatura",  
  "createdAt": "2026-06-12T15:00:00Z"  
}
```

Particionamento

A chave da mensagem será o `transactionId`.

```
key = transactionId
```

Isso garante ordenação por transação.

Fase 7 — Pix Processor

O `pix-processor` será responsável por:

1. Consumir eventos `pix.requested`
2. Buscar a transação no banco
3. Verificar se ainda está `PROCESSING` ou `RETRYING`
4. Chamar a instituição financeira parceira
5. Atualizar status para `COMPLETED` ou `FAILED`

Fluxo

```
PixRequested recebido  
|  
v  
Carrega transação
```

```
|
v
Status ainda processável?
|
+-- não --> ignora
|
+-- sim
    |
    v
    chama parceiro
    |
    +-- sucesso --> COMPLETED
    |
    +-- erro temporário --> RETRYING
    |
    +-- erro definitivo --> FAILED
```

Fase 8 — Integração externa com WireMock

O WireMock simulará a instituição financeira parceira.

Ele ficará fora do `pix-processor`, como dependência externa no Docker Compose.

Cenários simulados

```
200 OK
500 Internal Server Error
408 Timeout
429 Too Many Requests
Latência fixa de 2 segundos
Resposta inválida
```

Exemplo

```
stubFor(post("/partner/pix")
    .willReturn(aResponse()
        .withFixedDelay(2000)
        .withStatus(200)
        .withHeader("Content-Type", "application/json")
        .withBody("""
            {
                "partnerReferenceId": "partner-123",
                "status": "APPROVED"
            }
            """))));
```

Fase 9 — Resiliência com Resilience4j

O `pix-processor` usará:

```
Timeout
Retry
Circuit Breaker
Rate Limiter opcional
Bulkhead opcional
```

Retry

Usado para falhas temporárias.

Exemplo:

```
HTTP 500
HTTP 502
HTTP 503
HTTP 504
Timeout
```

Circuit Breaker

Evita sobrecarregar o parceiro quando ele está instável.

Estados:

```
CLOSED
OPEN
HALF_OPEN
```

Configuração sugerida

```
resilience4j:
  circuitbreaker:
    instances:
      partnerPix:
        slidingWindowSize: 20
        failureRateThreshold: 50
        waitDurationInOpenState: 30s
        permittedNumberOfCallsInHalfOpenState: 5
```



```
retry:
  instances:
    partnerPix:
      maxAttempts: 3
      waitDuration: 1s

timelimiter:
  instances:
    partnerPix:
      timeoutDuration: 3s
```

Trade-off

Retries aumentam chance de sucesso, mas também podem amplificar carga no parceiro. Por isso devem ser combinados com backoff, circuit breaker e limite de tentativas.

Fase 10 — Observabilidade com Datadog

Logs estruturados

Todos os logs devem conter:

```
transactionId
correlationId
status
service
attempt
errorCode
```

Métricas

```
pix.api.requests
pix.api.errors
pix.transactions.created
pix.transactions.completed
pix.transactions.failed
pix.processor.consumed
pix.processor.retries
pix.processor.dlq
pix.partner.latency
pix.partner.errors
pix.outbox.pending
pix.outbox.published
```

Traces

Fluxo rastreável:

```
POST /pix
-> PostgreSQL insert
-> Outbox event
-> Kafka publish
-> Kafka consume
-> Partner HTTP call
-> PostgreSQL update
```

Alertas sugeridos

```
Taxa alta de FAILED
Aumento de mensagens na DLQ
Latência alta no parceiro
Circuit breaker aberto por muito tempo
Outbox acumulando eventos pendentes
Consumer lag elevado no Kafka
```

Fase 11 — Docker Compose

O ambiente local deve subir com:

```
pix-api
pix-processor
postgres
kafka
redis
wiremock
datadog-agent
```

Comando esperado

```
docker compose up --build
```

Fase 12 — Testes

Testes unitários

```
PixService  
IdempotencyService  
OutboxService  
PartnerClient  
PixProcessor
```

Testes de integração

```
POST /pix cria transação  
POST /pix duplicado não duplica  
GET /pix retorna status  
Outbox publica evento  
Processor consome evento  
Processor atualiza status  
Partner com erro gera retry  
Partner indisponível abre circuit breaker
```

Teste end-to-end

```
1. POST /pix  
2. API retorna PROCESSING  
3. Evento é publicado  
4. Processor consome  
5. WireMock responde sucesso  
6. Status vira COMPLETED  
7. GET /pix retorna COMPLETED
```

Fase 13 — Estrutura do repositório

```
pix-platform/  
  docker-compose.yml  
  README.md  
  docs/  
    architecture.md  
    trade-offs.md  
    observability.md  
  pix-api/  
    src/main/java/...
```

```
Dockerfile
pix-processor/
  src/main/java/...
  Dockerfile
wiremock/
  mappings/
  __files/
```

Fase 14 — Decisões técnicas

Decisão 1 — Processamento assíncrono

A API não espera o parceiro responder.

Motivo:

```
Reduz latência percebida
Melhora escalabilidade
Permite retry controlado
Evita prender threads HTTP
```

Trade-off:

```
Cliente recebe PROCESSING e precisa consultar status
Sistema passa a ter consistência eventual
```

Decisão 2 — Kafka

Kafka será usado para desacoplar recebimento e processamento.

Motivo:

```
Suporta alto volume
Permite particionamento
Permite replay
Reduz acoplamento entre API e processor
```

Trade-off:

```
Mais complexo que uma fila simples
Exige cuidado com consumer lag, particionamento e operação
```

Decisão 3 — PostgreSQL como fonte da verdade

O estado da transação fica no PostgreSQL.

Motivo:

- Consistência transacional
- Facilidade de consulta
- Controle de idempotência
- Auditoria

Decisão 4 — Redis como cache auxiliar

Redis será usado para otimizar idempotência e consulta de status em alto volume.

Motivo:

- Reduz leituras repetidas no banco
- Melhora tempo de resposta

Trade-off:

- Cache pode ficar temporariamente desatualizado
- Não deve ser fonte da verdade

Decisão 5 — Outbox Pattern

Usado para garantir confiabilidade entre banco e Kafka.

Motivo:

- Evita perder eventos
- Mantém consistência entre transação criada e evento publicado

Trade-off:

- Mais uma tabela
- Mais um job/processo de publicação
- Maior complexidade operacional

Fase 15 — Critérios de aceite

Funcionais

```
POST /pix recebe uma transação válida
POST /pix retorna 202 Accepted
GET /pix/{transactionId} retorna status atual
Transações duplicadas não são reprocessadas
Processor atualiza status após resposta do parceiro
```

Não funcionais

```
API não bloqueia aguardando parceiro
Sistema suporta múltiplos consumers
Falhas temporárias geram retry
Falhas persistentes vão para DLQ
Logs, métricas e traces são enviados ao Datadog
Ambiente local sobe via Docker Compose
```

Fase 16 — Roadmap de desenvolvimento

Etapas 1

Criar estrutura dos projetos:

```
pix-api
pix-processor
docker-compose
```

Etapas 2

Implementar banco:

```
pix_transaction
outbox_event
migrations com Flyway ou Liquibase
```

Etapas 3

Implementar `pix-api`:

```
POST /pix
GET /pix/{transactionId}
idempotência
persistência
outbox
```

Etapa 4

Implementar publicação Kafka:

```
OutboxPublisher
KafkaProducer
pix.requested
```

Etapa 5

Implementar `pix-processor` :

```
KafkaConsumer
PartnerClient
status update
retry
circuit breaker
DLQ
```

Etapa 6

Implementar WireMock:

```
sucesso
erro temporário
timeout
latência de 2 segundos
```

Etapa 7

Implementar observabilidade:

```
logs estruturados
métricas
tracing
Datadog Agent
```

Etapa 8

Criar README final:

```
como subir  
como testar  
decisões técnicas  
trade-offs  
cenários de falha  
possíveis evoluções
```

Conclusão

A solução proposta reduz o impacto da latência externa ao remover a chamada ao parceiro do caminho crítico da API.

A arquitetura usa processamento assíncrono com Kafka, persistência confiável com PostgreSQL, idempotência por `transactionId`, cache auxiliar com Redis, resiliência com Resilience4j e observabilidade com Datadog.

Essa abordagem prepara a plataforma para crescimento de volume, melhora a disponibilidade percebida e permite recuperação controlada em cenários de falha temporária.